

Python Course for Physicists (And everyone else!)

Chunk 10: 2D numpy and mpl

By Jonathan Bollig

09.10.2025

Nested Numpy Arrays:

- Possible, if we have a homogenous shape:
 - All “sub-lists” have the same length
 - 1D is a vector, 2D is a Matrix, etc.
- Arbitrary dimensions possible
 - shape: dimensions & lengths



```
[[1, 2, 3],  
 [1, 2, 3]]
```

```
[[1, 2],  
 [1, 2],  
 [1, 2]]
```

```
[[[1, 2], [1, 2]],  
 [[1, 2], [1, 2]]]
```



```
[[1, 2, 3],  
 [1, 2]]
```

Live Coding: Creating nD Numpy Arrays

- Via List:

```
import numpy as np
my_list = [[1, 2, 3],
           [1, 2, 3]]
my_array = np.array(my_list)

my_array2 = np.array([[4, 5, 6],
                     [4, 5, 6]])
```

- Via functions:

- `np.zeros(shape)` / `np.ones(shape)`: Array filled with 0 / 1 of shape “shape”.
- `np.full(n, what)`: Array of shape “shape”, filled with “what”

The Shape

- Shape is a tuple, indicating the length of every dimension / axis

Array:

```
[[1, 2, 3],  
 [1, 2, 3]]
```



Shape:

(2, 3)

```
[[1, 2],  
 [1, 2],  
 [1, 2]]
```



(3, 2)

```
[[[1, 2], [1, 2]],  
 [[1, 2], [1, 2]]]
```



(2, 2, 2)

- Shape can be obtained by printing

```
my_list = [[1, 2, 3],  
           [1, 2, 3]]  
my_array = np.array(my_list)  
print(my_array.shape)
```

(2, 3)

Live Coding: Indexing

- Same as before, but dimensions can be separated by coma:
- Absence of coma indicates first dimension:
- With a coma, the next dimension is accessed:

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])  
print(my_array[0])  
[1 2 3]
```

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])  
print(my_array[0, 0])  
1
```

Live Coding: Slicing

- Slicing: Still used to select a range:
Here: First index selects 2. row.
Second index selects last 2 elements

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])  
print(my_array[1, 1:])
```

```
[5 6]
```

- Passing no numbers before and after “:” returns everything:

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])  
print(my_array[:, :])
```

```
[[1 2 3]  
 [4 5 6]]
```

- How to extract [3, 6]? Select all rows and the last column:

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])  
print(my_array[:, -1])
```

```
[3 6]
```

Live Coding: Useful Functions

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])
```

- Transpose: .T

```
print(my_array.T)
```

```
[[1 4]  
 [2 5]  
 [3 6]]
```

- Flatten: .flatten()

```
print(my_array.flatten())
```

```
[1 2 3 4 5 6]
```

- Concatenate: np.concatenate((tuple of arrays))

```
my_array = np.array([[1, 2, 3],  
                     [4, 5, 6]])  
print(np.concatenate((my_array, my_array)))
```

```
[[1 2 3]  
 [4 5 6]  
 [1 2 3]  
 [4 5 6]]
```

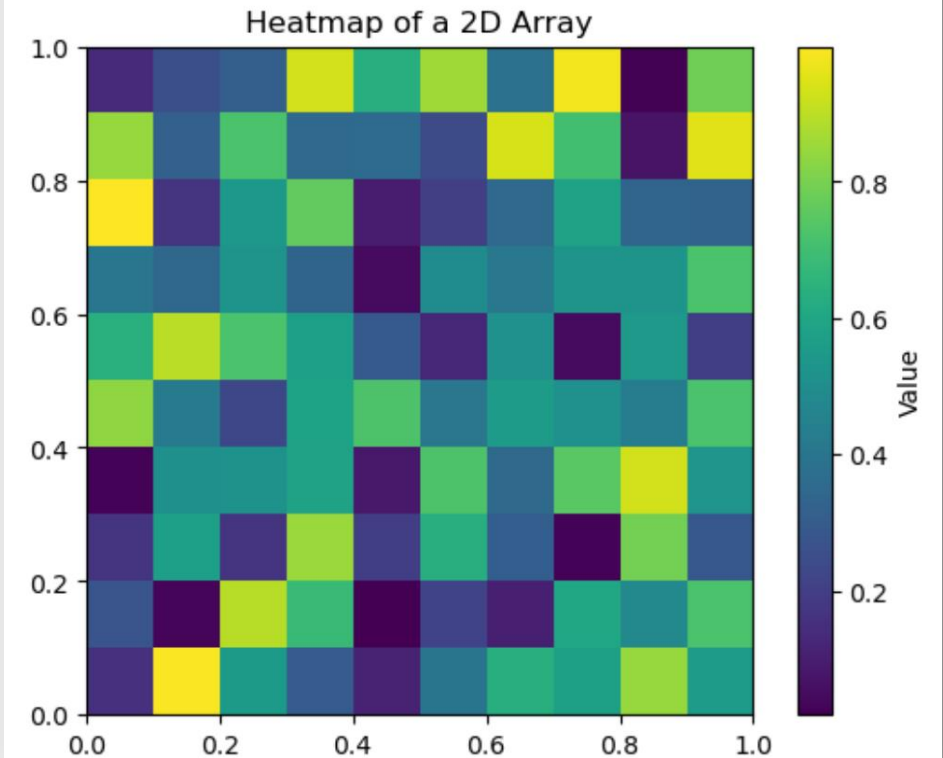
- A ton more functions can be found [here](#).

Live Coding: 2D – Plots

- `imshow()` plots an image of a 2D array:
 - The color of a pixel represents the value.
 - The optional argument “`extent`” takes a tuple `(x_min, x_max, y_min, y_max)` the defines the x and y-scale. Default are pixel.
- `colorbar()` adds the scale on the side.

```
random_matrix = np.random.random((10, 10))

plt.imshow(random_matrix, extent=(0, 1, 0, 1))
plt.title("Heatmap of a 2D Array")
plt.colorbar(label="Value")
plt.show()
```



Good Luck with your Tasks 😊



**learning
numpy axis
rules**

**print output
array's
shape until
one of the
the axis
values
works out**

[<https://andrewm4894.com/2020/10/15/numpy-feature-engineering-2x-speed-up-over-pandas/>]